ChargeSquare

Data Engineer - Case Study

EV Charging Data Pipeline — Real-Time & Analytics

1. Project Overview

ChargeSquare is a technology company operating in the EV charging infrastructure space, aggregating multiple charging networks into a single platform. The platform connects charging station operators (CPOs), eMobility service providers (eMSPs), and vehicle owners, generating real-time data streams.

In this case study, you are expected to design and implement a data engineering pipeline capable of processing the charging session events produced by our platform. Your task is to build an end-to-end system covering data generation, transport, storage, and analytics.

Core Objective

Write a simulator that generates 10,000–100,000 events per second from charging stations; transport these events via Kafka; write real-time data to a fast data store and long-term data to an analytics store; and derive meaningful insights from the data.

2. System Context & Data Model

2.1 What is a Charging Event?

You will use an event format inspired by OCPI (Open Charge Point Interface) and OCPP (Open Charge Point Protocol) standards, but not a verbatim copy. During each charging session, various event types flow into the system:

Event Type	Frequency	Description
SESSION_START	Low	Vehicle plugged in, session started
METER_UPDATE	High (every 5–30 sec)	Instantaneous power (kW), energy transferred (kWh), voltage, current
STATUS_CHANGE	Medium	Station status change (Available, Charging, Faulted, etc.)
SESSION_STOP	Low	Session ended; total cost and energy info
HEARTBEAT	Medium (every 30 sec)	Station liveness signal
FAULT_ALERT	Rare	Error code, component info

2.2 Suggested Event Schema

The following JSON structure is provided as a reference; the developer may extend or revise it as needed.

```
{
  "event_id":    "uuid-v4",
  "event_type":  "METER_UPDATE",
  "station_id":  "TR-IST-0042",
  "connector_id": 2,
  "session_id":  "sess-8f3a...",
  "timestamp":   "2025-06-15T14:23:07.412Z",
  "operator_id": "ChargeSquare",
  "location": {
    "lat": 41.0082, "lon": 28.9784,
    "city": "Istanbul", "country": "TR"
  },
  "meter": {
    "power_kw": 42.5, "energy_kwh": 18.3,
    "voltage_v": 400, "current_a": 106.25,
    "soc_percent": 67
  },
  "vehicle": {
    "brand": "Tesla", "model": "Model 3",
    "ev_id": "WVWAA7AU0KW123456"
  },
  "tariff_id":  "peak-rate-v2",
  "cost_eur":   4.28
}
```

2.3 Scale & Requirements

The system is expected to operate at the following scale:

- 10,000 – 100,000 events/second production capacity
- Parallel consumption support
- Real-time data: queryable with < 1 second latency
- Long-term data: monthly/yearly aggregation queries must be supported
- At-least-once delivery guarantee: every event must be processed at least once; this means duplicate events may occur.

3. Tasks

Each task can be evaluated independently. For tasks you cannot complete, explain your design decisions and the challenges you encountered in writing.

Task 1 — Data Simulator

Write a data producer that simulates a charging station network.

Requirements:

1. Define at least 100 virtual stations, each with 1–4 connectors.
2. Model realistic session behavior: peak hours during the day (07:00–09:00, 17:00–20:00), charge duration distribution, station fault rate.
3. When SESSION_START is triggered, generate consecutive METER_UPDATE events; send SESSION_STOP when the session ends.
4. Allow station count and event rate to be configured via a configuration file (YAML/JSON).
5. Forward produced events to the pipeline via file (JSON/CSV/Parquet) or any method of your choice.

> **Tip**
> Any language is permitted (Python, Go, Java...). If performance is a priority, async I/O or a thread pool is recommended. The simulator does not need to be perfect; focus on data quality and realism.

Task 2 — Data Pipeline

Build a pipeline that reads the produced events, transforms them, and writes to two different destinations.

> **Kafka is Optional**
> Using Kafka as the transport layer is optional. A file-based or HTTP-based pipeline is fully sufficient for this task. If you integrate Kafka, it will be counted as bonus points.
> Interview questions Q5–Q7 cover the Kafka scenario regardless of your implementation choice.

2a. Data Reading, Validation & Deduplication:

- Read events from the simulator (file, HTTP endpoint, or another method of your choice).
- Deserialize events and perform schema validation (missing fields, invalid values).
- Route invalid events to a separate ‘dead-letter’ file or table.
- Due to the at-least-once guarantee, the same event may arrive more than once. Design and implement a strategy for detecting and filtering duplicate events.
  - Which field(s) would you use as a unique key? (event_id, session_id + timestamp, etc.)

- Where and for how long would you store deduplication state?
- How would you handle late-arriving duplicates?

2b. Late & Out-of-Order Event Handling:

- METER_UPDATE events may arrive out of order or late due to network delays or retry mechanisms.
- When processing events, which time would you base your logic on: the time the event was produced (event-time) or the time it arrived at your pipeline (processing-time)? Explain your reasoning.
- When computing over a specific time window (e.g., 1-minute energy consumption sum), how would you handle late-arriving events?
  - How long would you keep the window open?
  - Where would you write late events that you do not accept?
  - Would you update an already-published window result?

2c. Real-Time Store:

- Maintain current station states (active sessions, instantaneous power, connection status) in a fast-access data store.
- Suggested technologies: Redis, Apache Cassandra, TimescaleDB, InfluxDB — state your preference and reasoning.
- A station's current status (last 5 minutes) must be queryable in under 100ms.

2d. Analytics Store:

- Write raw events and pre-aggregated data to a separate store for long-term retention.
- Suggested technologies: ClickHouse, Apache Parquet + DuckDB, BigQuery, Redshift — state your preference and reasoning.
- Define a partition strategy (event_date, operator_id, etc.).

Architecture Note

You may feed both stores from the same pipeline, or design them as separate processes.
Discuss the trade-offs of both approaches in your README.

Task 3 — Analytical Queries & Reporting

Run the following queries / analyses on the analytics store. Present results as tables or charts.

#	Analysis	Expected Output
A1	Hourly energy consumption (kWh) — last 7 days	Time series, peaks, low-usage hours
A2	Station-level uptime / downtime ratio	Most problematic stations per operator
A3	Average charging duration & energy amount — by vehicle brand	Tesla vs. others comparison

A4	Revenue analysis — tariff × energy × time	Peak-hour revenue contribution (%)
A5	Geographic distribution of FAULT events	Error density by city/region
A6	(Bonus) Anomaly detection	Sessions deviating >2σ from average power

Task 4 — Performance & Scalability

Observe and report how your system behaves under load.

- Scale the simulator incrementally: 1,000 → 10,000 → 50,000 → 100,000 events/sec.
- At each level, measure pipeline throughput, ingestion lag (time from event production to store write), and query durations.
- Identify bottlenecks and present your proposed solutions.
- (Bonus) Write a simple benchmark test for any one component.

Task 5 — Documentation & Architecture Report

Document your technical decisions and architectural choices.

- System architecture diagram (text-based ASCII is accepted; draw.io/Mermaid preferred).
- Justification for technology choices (why this DB, why this serialization format, etc.).
- Setup and run instructions (README.md).
- A step-by-step list of what you would need to do to move to a production environment.
- Known limitations and potential future improvements.

4. Evaluation Criteria

Criterion	What We Look For	Weight
Pipeline Design	Event flow logic, fault tolerance, at-least-once guarantee	25%
Data Model & Schema	Field selection, type consistency, partition strategy	20%
Store Selection & Justification	Understanding of real-time vs. analytics trade-offs	20%
Analytical Queries	Query correctness, performance awareness, interpretation quality	15%
Code Quality	Readability, modularity, comments	10%

Documentation	Clarity of architecture, README completeness, decision justifications	10%

⚠️ **Important Note**

Completing everything 100% is not expected. Explaining what you would do with incomplete parts and why you prioritized as you did is more valuable than completed work without explanation.

5. Technology Recommendations

*The following are **recommendations**, not requirements. If you choose a different technology, state your reasoning.*

Layer	Suggested Options	Notes
Data Transport	File (JSON/Parquet), HTTP API, PostgreSQL COPY	Choice is free; state your reasoning
Data Transport (Bonus)	Kafka	Optional — earns bonus points if integrated
Serialization	JSON (start), Parquet/Avro (bonus)	Columnar format earns bonus points
Real-Time Store	Redis Streams, TimescaleDB, InfluxDB, Cassandra	Low latency is the priority
Analytics Store	ClickHouse, DuckDB+Parquet, BigQuery (free tier)	Columnar store preferred
Orchestration	Docker Compose (minimum), Airflow/Prefect (bonus)	All services must start with docker-compose
Visualization	Grafana, Jupyter Notebook, Metabase	SQL results as tables are also accepted
Programming Language	Java , Python (pandas, kafka-python, asyncio), Go	Python preferred but not required

6. Deliverables

6. GitHub / GitLab repository link (if private, access will be requested via invitation).
7. docker-compose.yml that brings up all services — the entire system must run with a single command (docker compose up).
8. README.md: setup, run, and test instructions.
9. Analytical query results (screenshot, notebook, or CSV).
10. Architecture decision report (2–5 pages, PDF or Markdown).
11. (Bonus) A simple Grafana / Metabase dashboard.

Duration

5–7 days are recommended for this case study. Our expectation is not a perfect product;
we want to see your engineering thinking, prioritization ability, and technical competency.

Looking Forward

Strategizing, Architecting, and Building—together, on repeat.

Good Luck! 🚀